

Respostas Exercícios OAC

RISC-V Uniciclo

1. a

Branch	MemRead	MemtoReg	ALUop	MemWrite	ALUSrc	RegWrite
0	0	0	10	0	0	1

1. b

Branch	MemRead	MemtoReg	ALUop	MemWrite	ALUSrc	RegWrite
0	1	1	00	0	1	1

2.a PC, Add4, MuxPC, MI, XREGS, Control, MuxUla, ULA, ULAControl, MuxReg

2.b PC, Add4, MuxPC, MI, XREGS, Control, ImmGen, MuxUla, ULA, ULAControl, MD, MuxReg

3.a ImmGen, Shift Left, Add Offset, MD

3.b Shift Left, Add Offset

4. Caminho crítico AND:

$$MI + XREGS (rd) + MuxULA + ULA + MuxReg + XREGS (wr)$$

a. Latência = $400 + 200 + 30 + 120 + 30 + 200 = 980$ ps

b. Latência = $500 + 220 + 100 + 180 + 100 + 220 = 1320$ ps

5. Caminho crítico LW:

$$MI + XREGS (rd) + MuxULA + ULA + MD + MuxReg + XREGS (wr)$$

a. Latência = $400 + 200 + 30 + 120 + 350 + 30 + 200 = 1330$ ps

b. Latência = $500 + 220 + 100 + 180 + 350 + 100 + 220 = 1670$ ps

6. Caminho crítico BEQ:

$$MI + XREGS (rd) + MuxULA + ULA + MuxPC$$

a. Latência = $400 + 200 + 30 + 120 + 30 = 780$ ps

b. Latência = $500 + 220 + 100 + 180 + 100 = 1100$ ps

7. Período do Relógio uniciclo (LW):

a. 1330 ps

b. 1670 ps

Ideal:

a. Log-Arith: 980 ps; LW: 1330 ps; SW: 1070 ps; BEQ: 780 ps

$$P_{ideal} : 980 \times 0,50 + 1330 \times 0,25 + 1070 \times 0,15 + 780 \times 0,10 = 1061 \text{ ps}$$

$$\text{Ganho} = 1330 / 1061 = 1,25 \Rightarrow 25\% \text{ mais rápido}$$

b. Log-Arith: 1320 ps; LW: 1670 ps; SW: 1250 ps; BEQ: 1100 ps

$P_{ideal} : 1320 \times 0,50 + 1670 \times 0,25 + 1250 \times 0,15 + 1100 \times 0,10 = 1375 \text{ ps}$
Ganho : $1670 / 1375 = 1,21 \Rightarrow 21\%$ mais rápido.

8.a $P = 400 + 200 + 30 + 300 + 350 + 30 + 200 = 1510 \text{ ps}$

8.b Log-Arith: 1160 ps; LW: 1510 ps; SW: 1250 ps; BEQ: 960 ps (p/ULA 300ps)

$P_{ideal} = 1160 \times 0,50 + 1510 \times 0,25 + 1250 \times 0,15 + 960 \times 0,10 = 1241 \text{ ps}$

Ganho: $T_a / T_{ideal} = I_a \times CPI_a \times P_a / 0,95 I_a \times CPI_{ideal} \times P_{ideal}$

O ideal também é uniciclo, portanto, $CPI_{ideal} = 1$

Ganho: $1/0,95 \times P_a / P_{ideal} = 1/0,95 \times 1510/1241 = 1,28 \Rightarrow 28\%$ mais rápido.

11. Apenas o LW.

12. As instruções afetadas pela falha seriam as lógico-aritméticas com dado imediato (incluindo LUI e AUIPC), *load* e *store* (usa o imediato para calcular o endereço).

RISC-V Multiciclo

RISC-V Pipeline

1. Pipeline: 350; Unicycle: 1250

2. Pipeline: 1750; Unicycle: 1250

3. Estágio Decode. O novo clock teria período de 300 ps.

4. 35%

5. $n + k - 1$. A primeira instrução leva k ciclos, as restantes um ciclo para cada, ou $n-1$ ciclos, considerando que uma instrução está completa quando passa pelo último estágio do *pipeline*.

6. $x13 = 33$ e $x14 = 26$

7. $x15 = 54$

8. `addi x11, x12, 5`
 `NOP`
 `NOP`
 `NOP`
 `add x13, x11, x12`
 `addi x14, x11, 15`
 `NOP`

NOP
add x15, x13, x12

9. O *pipeline* sem adiantamento requer $1,4 * n * 250$ ps. A versão com adiantamento requer $1,05 * n * 300$ ps. A aceleração é $(1,4 * 250) / (1,05 * 300) = 1,11$.

10. A relação $(1,4 * 250)/(x*300) = 1$ indica um fator $x = 1,167$, ou seja, se tivermos mais de 16,7% de NOPs no *pipeline* com adiantamento, a sobrecarga introduzida pelo adiantamento resulta em um desempenho inferior.

11. O período do relógio não se altera, pois o atraso do estágio de execução permanece inferior ao atraso do estágio de decodificação, que determina o período.

12. Antecipando a leitura da memória para o estágio de execução elimina o conflito de *load* seguido de instrução usa o dado lido da memória, o que vai eliminar bolhas no fluxo de execução.

13. O cálculo do endereço em algumas instruções terá que ser realizado por outros instruções, aumentando o tamanho do código.

14. O diagrama (b), pois as instruções em IF e ID permanecem congeladas enquanto a instrução LW progride para o estágio MD, impedindo a busca de uma nova instrução, como ocorre em (a).

15. No diagrama apresentado, o salto condicional não ocorre se (1) a instrução é BEQL e o resultado da comparação entre os registradores na saída do banco é falso. Isso implica nas seguintes modificações:

- a) Controle gera um sinal *is_beql*, que identifica a instrução
- b) Uma porta AND é introduzida, tendo como entradas *is_beql* e zero (gerado pelo comparador na saída do banco de registradores)
- c) A lógica de geração de bolhas, i. e., o sinal de seleção do multiplexador que fornece os sinais de controle ao ID/EX, é alterada para fornecer zero caso a condição acima é falsa, ou seja, o salto não ocorre. Por exemplo, o sinal de seleção pode ser conectado a uma porta OR, cujas entradas seriam o sinal da Hazard Detection Unit ou a negação do sinal da condição de salto, supondo o valor zero como entrada da condição $sel == 1$.